Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Modern C++ ve STL Uygulamaları

Bölüm 3 – Hafta 3

Yazılım Geliştirme II (BILB2000)

Bahar Dönemi – 3. Hafta

Bu Haftanın Planı

1. Ünite 1: Ranges ve Views (C++20)
2. Ünite 2: String İşlemleri
3. Ünite 3: Hata Yönetimi
4. Ünite 4: OBYS STL Dönüşümü
5. Özet

Ünite 1: Ranges ve Views (C++20)

Neden Ranges?

Eski Yöntemin Sorunu

İki iteratör ayrı ayrı belirtilir → uyumsuz çift hatası mümkün!

```
// TEHLIKE: Farkli kapsayicilardan iterator cifti!
std::sort(v1.begin(), v2.end(), ...);
// Derleyici hata VERMEZ, program cokebilir!
```

- Her algoritma çağrısında `begin()/end()` yazmak gereksiz tekrar
- Yanlış kapsayıcıdan iteratör almak sessiz hata kaynağı
- C++20 **ranges** kütüphanesi bu sorunu kökten çözer

Ranges'ın Üç Vaadi

Güvenlik (uyumsuz çift yok) **Okunabilirlik** (daha kısa kod) **Composability** (pipeline)

Range Kavramı

Tanım

Range (aralık), `begin()` ve `end()` fonksiyonlarına sahip olan herhangi bir nesnedir. Üzerinde baştan sona dolaşılabilecek bir eleman dizisidir.

Range olan yapılar:

- `std::vector`, `std::string`, `std::map`
- `std::array`, `std::list`, `std::set`
- C-dizileri (`int arr[5]`)
- `std::views` tarafından üretilen tembel görünümler

Kural

`begin()` + `end()` = Range. Bu kadar basit!

ranges::sort vs std::sort

Eski Yöntem:

```
#include <algorithm>

std::vector<int> notlar =
    {78, 95, 62, 88, 45};

// Iki iterator belirt
std::sort(
    notlar.begin(),
    notlar.end());
```

Yeni Yöntem (C++20):

```
#include <algorithm>
#include <ranges>
std::vector<int> notlar =
    {78, 95, 62, 88, 45};

// Dogrudan kapsayiciyi ver
std::ranges::sort(notlar);
```

Avantaj

`ranges::sort(notlar)`: Kapsayıcıyı bir bütün olarak veriyorsun → yanlış iteratör çifti hatası **imkansız**.

ranges::find_if, count_if ve any_of

```
struct Ogrenci { std::string isim; int numara; double ort; };
std::vector<Ogrenci> sinif = {
    {"Ali", 1001, 3.2}, {"Zeynep", 1002, 3.8},
    {"Mehmet", 1003, 2.9}, {"Ayse", 1004, 3.5}
};

// Dogrudan kapsayici ile arama
auto it = std::ranges::find_if(sinif,
    [](const Ogrenci& o) { return o.numara == 1003; });

// Basarili ogrenci say
auto basarili = std::ranges::count_if(sinif,
    [](const Ogrenci& o) { return o.ort >= 3.0; });

// Herhangi biri onur listesinde mi?
bool var = std::ranges::any_of(sinif,
    [](const Ogrenci& o) { return o.ort >= 3.9; });
```

Header

`#include <ranges>` + `-std=c++20` derleme bayrağı gerekli.

Range Concept'leri

Concept	Gereksinim	Örnek Kapsayıcı	İteratör Dengi
`input_range`	Tek geçiş okuma	`istream_view`	Input
`forward_range`	Çok geçişli okuma	`forward_list`	Forward
`bidirectional_range`	İleri + geri	`list`, `map`	Bidirectional
`random_access_range`	Rastgele atlama	`deque`	Random Access
`contiguous_range`	Ardışık bellek	`vector`, `string`	Contiguous
`sized_range`	$O(1)$ boyut	`vector`, `map`	–

Derleme Zamanı Kontrolü

Algoritma gerektirdiği concept'i sağlamayan bir range alırsa $\rightarrow$ anlaşılır bir **derleme hatası** verir. Eski STL'deki sayfalarca template hata mesajlarına elveda!

Eski vs Yeni: Diyagram

Eski: Iterator-Pair

```
auto it = std::find_if(
 v.begin(), v.end(),
 [](int x){ return x > 5; });
std::sort(v.begin(), v.end());
```

Başlangıç ve bitiş ayrı ayrı belirtilir

VS

daha güvenli

daha okunabilir

Yeni: Ranges

```
auto it = std::ranges::find_if(
 v,
 [](int x){ return x > 5; });
std::ranges::sort(v);
```

Kapsayıcı doğrudan geçilir

Eski:

- İki ayrı iteratör belirt
- Uyumsuz çift riski
- Uzun söz dizimi

Yeni (Ranges):

- Kapsayıcıyı doğrudan ver
- Derleme zamanı güvenlik
- Kısa ve okunabilir

Projeksiyonlar

Tanım

Projeksiyon: sıralama/arama öncesinde her elemana uygulanacak dönüşüm. Üye değişken veya fonksiyon işaretçisi olabilir.

```
struct Ogrenci { std::string isim; int numara; double ort; };
std::vector<Ogrenci> sinif = { /* ... */ };

// ESKI: lambda ile karsilastirici (uzun)
std::sort(sinif.begin(), sinif.end(),
    [](const Ogrenci& a, const Ogrenci& b) {
        return a.numara < b.numara; });

// YENI: projeksiyon ile (kisa ve net)
std::ranges::sort(sinif, {}, &Ogrenci::numara);
// {} = varsayilan karsilastirici, &Ogrenci::numara = projeksiyon
```

Lambda'ya Veda

Tek üye değişkene göre sıralama/arama → projeksiyon kullan, lambda yazma!

Projeksiyon Örnekleri

```
struct Ogrenci { std::string isim; int numara; double ort; };
std::vector<Ogrenci> sinif = { /* ... */ };

// Numaraya gore ogrenci bul
auto it = std::ranges::find(sinif, 1002, &Ogrenci::numara);

// Ortalamasi 3.0+ ogrenci say (projeksiyon + kosul)
auto n = std::ranges::count_if(sinif,
    [](double ort) { return ort >= 3.0; },
    &Ogrenci::ort);

// Isme gore sirala (buyukten kucuge)
std::ranges::sort(sinif, std::greater{}, &Ogrenci::isim);

// En yuksek ortalamali ogrenci
auto en_iyi = std::ranges::max_element(sinif, {},
    &Ogrenci::ort);
```

Views Nedir?

Tanım

View (görünüm), bir range üzerinde **tembel** (lazy) ve **kopyalamadan** çalışan bir dönüşüm katmanıdır.

Klasik Yaklaşım:

1. Filtrele → yeni vector
2. Dönüştür → yeni vector
3. Sonucu kullan
4. 2 ek kopya, 2 bellek tahsisi

Views Yaklaşımı:

1. Tarifi tanımla (filter + transform)
2. for döngüsünde tüket
3. 0 kopya, 0 bellek tahsisi
4. Elemanlar talep üzerine hesaplanır

Tembel Değerlendirme (Lazy Evaluation)

View oluşturulduğunda hiçbir iş yapılmaz. Elemanlar yalnızca `for` döngüsü veya başka bir tüketici tarafından çekildiğinde hesaplanır.

filter ve transform

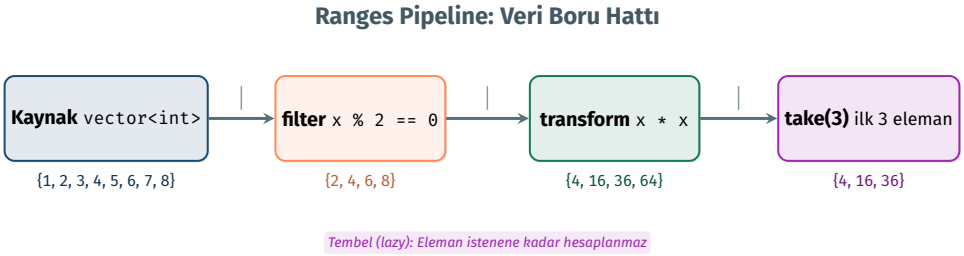
```
#include <ranges>
#include <vector>
#include <iostream>
std::vector<int> notlar = {45, 78, 92, 33, 85, 61, 55, 90};

// filter: sadece 60 ve ustu notlari gec (kopya yok!)
auto gecenler = notlar
    | std::views::filter([](int n) { return n >= 60; });

for (int n : gecenler)
    std::cout << n << " ";    // 78 92 85 61 90

// transform: her notu 10 uzerinden goster
auto onluk = notlar
    | std::views::transform([](int n) { return n / 10.0; });

for (double d : onluk)
    std::cout << d << " ";    // 4.5 7.8 9.2 3.3 8.5 6.1 5.5 9
```

take, drop, keys, values, iota

View	Açıklama	Örnek
`take(n)`	İlk n eleman	`v | views::take(3)`
`drop(n)`	İlk n'i atla	`v | views::drop(2)`
`keys`	map anahtarları	`m | views::keys`
`values`	map değerleri	`m | views::values`
`iota(a,b)`	$[a, b)$ sayı üret	`views::iota(1, 11)`
`reverse`	Ters sıra	`v | views::reverse`
`enumerate`	İndeksli (C++23)	`v | views::enumerate`

```
// Sayfalama: sayfa 3'teki 20 ogrenciyi goster
auto sayfa3 = ogrenciler | views::drop(40) | views::take(20);

// Map'ten sadece numaralari al
std::map<int, std::string> kayit = {{1001,"Ali"},{1002,"Ayse"}};
for (int no : kayit | std::views::keys)
    std::cout << no << " ";  // 1001 1002
```

Views Pipeline Diyagramı



Veri Akışı

Kaynak → `filter` → `transform` → `take` → Tüketici (for döngüsü)

Her aşamada veri kopyalanmaz. Yalnızca "ne yapılacağı" tarif edilir ve elemanlar tüketici tarafından çekildiğinde hesaplanır.

Pipe Operatörü (|)

Tanım

Pipe operatörü, soldaki range'i sağdaki view adaptörüne bağlar. Unix | ile aynı mantık.

```
std::vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// Ic ice fonksiyon cagrisi (ZOR OKUNUR!)
auto s1 = std::views::take(
    std::views::transform(
        std::views::filter(v,
            [](int x) { return x % 2 == 0; }),
        [](int x) { return x * x; }), 3);

// Pipe sozdizimi (OKUNAK!) -- ayni sonuc: {4, 16, 36}
auto s2 = v
    | std::views::filter([](int x) { return x % 2 == 0; })
    | std::views::transform([](int x) { return x * x; })
    | std::views::take(3);
```

OBYS: Ranges ile Sorgulama

```
struct Ogrenci {
    std::string isim; int numara; double ort; std::string bolum;
};
std::vector<Ogrenci> kayitlar = { /* ... */ };

// Once numaraya gore sirala
std::ranges::sort(kayitlar, {}, &Ogrenci::numara);

// Pipeline: Bilgisayar bolumu + basarili + isimleri al
auto rapor = kayitlar
    | std::views::filter([](const Ogrenci& o) {
          return o.bolum == "Bilgisayar"; })
    | std::views::filter([](const Ogrenci& o) {
          return o.ort >= 60.0; })
    | std::views::transform([](const Ogrenci& o) {
          return o.isim; });

for (const auto& isim : rapor)
    std::cout << "  " << isim << "\n";
```

Ünite 2: String İşlemleri

string_view Nedir?

Tanım

`std::string_view`: bir karakter dizisine **salt-okunur** ve **sahiplenmeyen** bir görünüm. İçinde yalnızca bir işaretçi (`const char*`) ve bir uzunluk (`size_t`) tutar.

std::string:

- Veriyi **sahiplenir**
- Kendi belleğini tahsis eder
- Kitabın **fotokopisi**

std::string_view:

- Veriyi **ödünç alır**
- Bellek tahsisi yok
- Kitabı **raftan okumak**

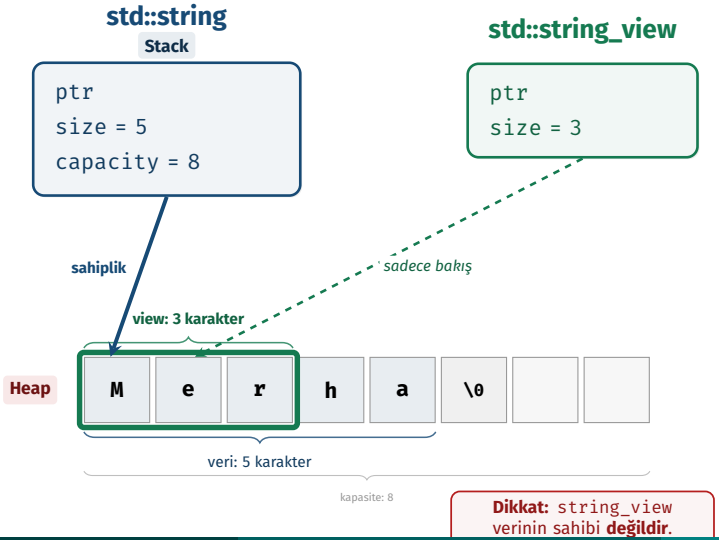
Motivasyon

`void ara(const std::string& s)` → literal geçince geçici string oluşur.
`void ara(std::string_view s)` → hiçbir kopya yok!

string_view Oluşturma ve İşlemler

```
#include <string_view>
#include <string>
// Farkli kaynaklardan olusturma
std::string_view sv1 = "Merhaba Dunya";   // literal'den
std::string isim = "Ali Yilmaz";
std::string_view sv2 = isim;               // string'den
std::string_view sv3 = sv1.substr(0, 7);   // "Merhaba"
// substr() kopya OLUSTURMAZ! Sadece gorunum daraltir

// Arama islemleri
auto pos = sv1.find("Dunya");         // 8
auto son = sv1.rfind("a");            // 12

// C++20: starts_with ve ends_with
std::string_view dosya = "OBYS_Rapor.pdf";
bool obys = dosya.starts_with("OBYS");  // true
bool pdf  = dosya.ends_with(".pdf");    // true

sv1.size();  // 13    sv1.front();  // 'M'    sv1.empty();  // false
```

string vs string_view Bellek



string_view Tehlikeleri

Sarkan Görünüm (Dangling View)

`string_view`'ın işaret ettiği verinin yaşam süresi biterse → **tanımsız davranış**!

```
// TEHLIKE 1: Gecici string'den string_view
std::string isim_olustur() { return "Ali"; }
std::string_view sv = isim_olustur(); // TANIMSIZ DAVRANIS!

// TEHLIKE 2: Kapsam disi
std::string_view sv2;
{
    std::string gecici = "Merhaba";
    sv2 = gecici;
}  // gecici yok edildi! sv2 sarkan
```

Altın Kural

`string_view`'ın ömrü, işaret ettiği verinin ömründen **kısa** olmalı.
En güvenli kullanım: **fonksiyon parametresi** olarak değer ile geçmek.

std::format Temelleri

Neden format?

cout → uzun, yapışkan manipülatörler. printf → tür güvensiz.
std::format → Python f-string tarzı, tür-güvenli, derleme zamanı kontrol.

```
#include <format>
int numara = 1001;
std::string isim = "Ali Yilmaz";
double gpa = 3.45;

// Temel kullanim: {} yer tutucular sirayla doldurulur
auto s1 = std::format("No: {} | Ad: {} | GPA: {}",
                      numara, isim, gpa);
// No: 1001 | Ad: Ali Yilmaz | GPA: 3.45

// Indeksli yer tutucular
auto s2 = std::format("{1} ({0}) - GPA: {2}",
                      numara, isim, gpa);
// Ali Yilmaz (1001) - GPA: 3.45
```

Format Belirteçleri

Belirteç	Açıklama	Örnek
{:<20}	Sola hizala, 20 genişlik		Ali Yilmaz	
{:>20}	Sağa hizala		Ali Yilmaz	
{:^20}	Ortala		Ali Yilmaz	
{:.2f}	2 ondalık basamak	3.14
{:08d}	Sıfırla doldur, 8 hane	00000042
{:x}	Hex gösterim	2a
{:b}	İkili gösterim	101010
{:+d}	İşaret göster	+42 / -42

```
// OBYS tablo satiri
std::cout << std::format(
    "{:>6} | {:<20} | {:>6.2f} | {:<15}\n",
    ogr.numara, ogr.isim, ogr.gpa, ogr.bolum);
// 1001 | Ali Yilmaz           |   3.45 | Bilgisayar
```

Özel Tür Formatlama

```
struct Ogrenci { int numara; std::string isim; double gpa; };
// std::formatter ozellistirmesi
template <>
struct std::formatter<Ogrenci> {
    constexpr auto parse(std::format_parse_context& ctx) {
        return ctx.begin();
    }
    auto format(const Ogrenci& o,
                std::format_context& ctx) const {
        return std::format_to(ctx.out(),
            "{}: {} (GPA: {:.2f})", o.numara, o.isim, o.gpa);
    }
};
// Artik dogrudan kullanabilirsin:
Ogrenci ogr{1001, "Ali Yilmaz", 3.45};
std::cout << std::format("Ogrenci: {}\n", ogr);
// Cikti: 1001: Ali Yilmaz (GPA: 3.45)
```

Genişletilebilirlik

Kendi sınıflarını doğrudan `std::format` ile yazdırabilirsin!

from_chars / to_chars

Neden?

stoi/to_string → locale bağımlı, istisna fırlatır. from_chars/to_chars → hızlı, locale-bağımsız, istisna yok.

```
#include <charconv>
#include <string_view>
// String'den sayiya donusum
std::string_view str = "12345";
int deger = 0;
auto [ptr, ec] = std::from_chars(
    str.data(), str.data() + str.size(), deger);
if (ec == std::errc{})
    std::cout << deger;  // 12345
else
    std::cout << "Hata: gecersiz giris";

// Sayidan string'e donusum
char buf[20];
auto [p, e] = std::to_chars(buf, buf + sizeof(buf), 42);
std::string_view sonuc(buf, p - buf);  // "42"
```

Veri Doğrulama: all_of, any_of, none_of

```
#include <algorithm>
#include <string_view>

// Tum notlar gecerli aralikta mi? (0-100)
std::vector<int> notlar = {85, 92, 78, 63, 95};
bool hepsi = std::all_of(notlar.begin(), notlar.end(),
    [](int n) { return n >= 0 && n <= 100; }); // true

// Ogrenci numarasi dogrulama
bool numara_gecerli(std::string_view numara) {
    if (numara.empty() || numara.size() != 6) return false;
    return std::all_of(numara.begin(), numara.end(),
        [](unsigned char c) { return std::isdigit(c); });
}

numara_gecerli("123456");   // true
numara_gecerli("12345a");   // false (harf var)
```

Doğrulama Üçlüsü

`all_of`: **hepsi** mi? `any_of`: **en az biri** mi? `none_of`: **hiçbiri** mi?

OBYS: String İşlemleri Entegrasyon

```
// 1. string_view ile kopyasiz parametre
const Ogrenci* ara(const std::vector<Ogrenci>& liste,
                   std::string_view aranan) {
    auto it = std::ranges::find_if(liste,
        [aranan](const Ogrenci& o) {
            return o.isim == aranan; });
    return it != liste.end() ? &(*it) : nullptr;
}

// 2. format ile tablo ciktisi
void tablo_yazdir(const std::vector<Ogrenci>& liste) {
    std::cout << std::format("{:^6} | {:<20} | {:>5}\n",
                             "No", "Isim", "GPA");
    std::cout << std::format("{:-^36}\n", "");
    for (const auto& o : liste)
        std::cout << std::format(
            "{:>6} | {:<20} | {:>5.2f}\n",
            o.numara, o.isim, o.gpa);
}
```

Ünite 3: Hata Yönetimi

Motivasyon: Arama Bulamadığında Ne Döner?

Eski Yaklaşımlar:

- `nullptr` → null dereference riski
- Boş nesne → "geçerli mi, boş mu?"
- Sihirli değer (-1) → gerçek veri sanılabilir
- Hata kodu → kontrol unutulabilir

Modern C++ Çözümleri:

- `std::optional` → "belki yok"
- `std::variant` → "birden fazla tür"
- İstisna hiyerarşisi → yapısal hata yönetimi

Temel Sorun

C++'ta "değer yok" durumunu güvenli şekilde ifade etmek uzun süre ciddi bir problem olmuştur. C++17 ile gelen `optional` ve `variant` bu sorunu çözer.

std::optional Temelleri

Tanım

std::optional<T>: ya bir T değer tutar ya da boştur (std::nullopt).

```
#include <optional>
// Degerli optional
std::optional<int> numara = 2001;
// Bos optional
std::optional<std::string> isim = std::nullopt;
std::optional<double> ortalama; // varsayilan: bos

// Bool donusumu: deger var mi?
if (numara)
    std::cout << "Numara: " << *numara << "\n";

// Erisim yollari
int n1 = *numara;                  // 2001 (tehlikeli: bossa UB!)
int n2 = numara.value();           // 2001 (bossa istisna)
int n3 = ortalama.value_or(0.0);   // 0.0 (varsayilan deger)
```

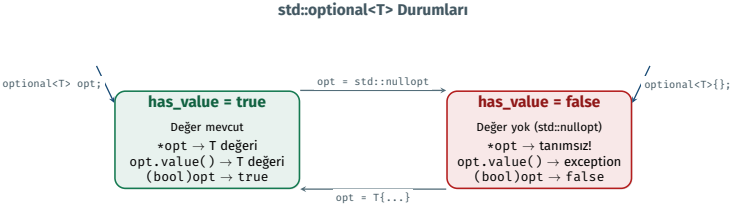
optional Kullanım Desenleri

```
struct Ogrenci { int numara; std::string isim; double ort; };

// Bulamazsa nullopt dondur -- fonksiyon imzasi net!
std::optional<Ogrenci> ogrenciBul(
    const std::vector<Ogrenci>& liste, int numara) {
    for (const auto& ogr : liste)
        if (ogr.numara == numara) return ogr;
    return std::nullopt;
}

// C++17 if-init sozdizimi ile kullanim
if (auto sonuc = ogrenciBul(kayitlar, 2001)) {
    std::cout << sonuc->isim << ": " << sonuc->ort << "\n";
} else {
    std::cout << "Ogrenci bulunamadi.\n";
}
```

Fonksiyon İmzası = Dokümantasyon

`optional<T>` döndüren fonksiyon "sonuç olmayabilir" mesajını **açıkça** verir.

optional Durumları



Dolu Durum:

- `has_value()` = *true*
- `*opt` ile erişim
- `opt->member` ile erişim

Boş Durum:

- `has_value()` = *false*
- `value()` → istisna!
- `value_or(x)` → x döner

std::variant Temelleri

Tanım

`std::variant<T1, T2, ...>`: belirtilen türlerden **tam olarak birini** tutan tür-güvenli birleşim (union).

```
#include <variant>
#include <string>
struct Muaf {};
// OBYS notu: sayi, harf veya muaf olabilir
std::variant<int, std::string, Muaf> not_degeri;

not_degeri = 85;                    // Sayi notu
not_degeri = std::string("AA");     // Harf notu
not_degeri = Muaf{};                // Muaf durumu
std::cout << not_degeri.index();    // 2 (Muaf)

// Guvenli erisim: get_if (nullptr dondurur)
if (auto* p = std::get_if<int>(&not_degeri))
    std::cout << "Sayi: " << *p << "\n";
```

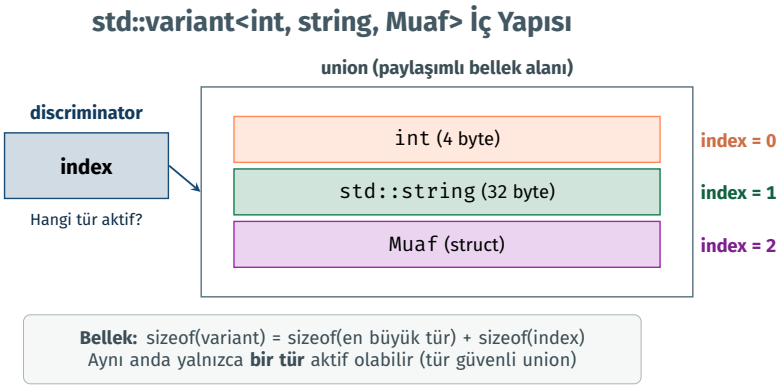
std::visit ve overloaded Deseni

```
// overloaded yardimci sablonu
template<class... Ts>
struct overloaded : Ts... { using Ts::operator()...; };

using NotTuru = std::variant<int, std::string, Muaf>;
NotTuru not_degeri = std::string("BA");

// Lambda'larla yerinde ziyaretci olustur
std::visit(overloaded{
    [](int sayi) {
        std::cout << "Sayi notu: " << sayi << "\n";
    },
    [](const std::string& harf) {
        std::cout << "Harf notu: " << harf << "\n";
    },
    [](Muaf) { std::cout << "Muaf\n"; }
}, not_degeri);
```

Derleme Zamanı Güvenlik

Bir tür için `operator()` tanımlamayı **unutursan** → derleme hatası!

variant Yapısı



variant
Tür-güvenli union
Her zaman birini tutar

optional
"Belki yok" durumu
Değer veya nullopt

any
Herhangi bir tür
Tip silme (erasure)

std::any – Kısa Tanıtım

```
#include <any>
std::any deger = 42;                // int tutar
deger = std::string("Merhaba"); // string tutar
deger = 3.14;                       // double tutar

// any_cast ile erisim
double d = std::any_cast<double>(deger);     // 3.14
// Yanlis tur -> std::bad_any_cast istisnasi!
```

Özellik	optional	variant	any
Tür sayısı	1 (veya boş)	Bilinen N tür	Herhangi biri
Tür güvenliği	✓	✓	Çalışma zamanı
Heap tahsisi	Hayır	Hayır	Olabilir
Kullanım alanı	"Belki yok"	"Birkaç türden biri"	Esneklik

Tavsiye

any yerine mümkünse `variant` tercih et. Derleme zamanı güvenlik!

İstisna Hiyerarşisi: std::exception

- `std::exception`
  - `logic_error` (programcı hatası)
    - `invalid_argument`
    - `out_of_range`
    - `domain_error`
  - `runtime_error` (çalışma zamanı)
    - `overflow_error`
    - `underflow_error`
    - `range_error`
  - `bad_optional_access` (C++17)
  - `bad_variant_access` (C++17)
  - `bad_any_cast` (C++17)

İki Ana Dal

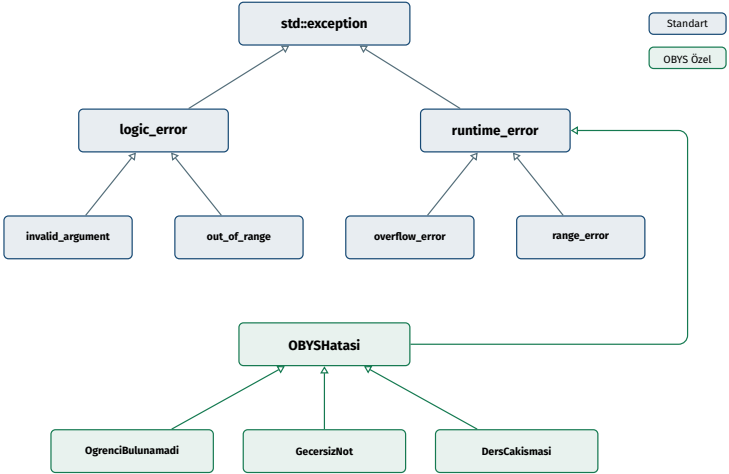
`logic_error`:
Programcı hatası → hiç olmamalı

`runtime_error`:
Çalışma zamanı hatası → olabilir

Özel İstisnalar: OBYS Hiyerarşisi

```
// Temel OBYS hatasi
class OBYSHatasi : public std::runtime_error {
public:
    explicit OBYSHatasi(const std::string& mesaj)
        : std::runtime_error(mesaj) {}
};

// Ogrenci bulunamadi
class OgrenciBulunamadi : public OBYSHatasi {
    int numara_;
public:
    explicit OgrenciBulunamadi(int numara)
        : OBYSHatasi("Ogrenci bulunamadi: "
                     + std::to_string(numara)),
          numara_(numara) {}
    int numara() const { return numara_; }
};

// Gecersiz not degeri
class GecersizNot : public OBYSHatasi {
    int deger_;
public:
    explicit GecersizNot(int deger)
        : OBYSHatasi("Gecersiz not: " + std::to_string(deger)),
          deger_(deger) {}
};
```

İstisna Hiyerarşisi Diyagramı



Yakalama Stratejisi

Hata Stratejisi Seçimi

Durum	Araç	Neden?	Örnek
Değer olmayabilir	`optional`	Hafif, kopya yok	Arama sonucu
Birkaç tür biri	`variant`	Tür-güvenli	Not türleri
Kurtarılabilir hata	İstisna	Çağrı yığını geri sarar	Dosya hatası
Programcı hatası	`assert`	Debug'da yakala	Ön koşul ihlali
Performans kritik	Hata kodu	İstisna maliyeti yok	Gerçek zamanlı

OBYS İçin Önerilen Yaklaşım

- Arama fonksiyonları → `optional` döndür
- Not sistemi → `variant<int, string, Muaf>`
- Beklenmedik hatalar → `OBYSHatasi` istisna hiyerarşisi

Ünite 4: OBYS STL Dönüşümü

Dönüşüm Planı: Üç Katman

Sistematik Refactoring

OBYS'yi rastgele değil, **üç katman** halinde modernize edeceğiz.

1. **Veri Katmanı**
   - `vector` → `map<int, Ogrenci>`
   - `nullptr` → `optional<Ogrenci>`
   - `double` → `variant<int, string, Muaf>`
2. **Servis Katmanı**
   - Ranges pipeline ile sorgulama
   - `string_view` parametreler, `format` çıktılar
3. **Hata Katmanı**
   - `OBYSHatasi` istisna hiyerarşisi
   - `OgrenciBulunamadi`, `GecersizNot`, `DersCakismasi`

Eski Mimari

Sorunlar

`vector` ile $O(n)$ arama · "Bulunamadı" = `nullptr` · Hata yönetimi yok, not türleri esnek değil

Veri Katmanı: vector → map, optional, variant

```
struct Muaf {};
using NotTuru = std::variant<int, std::string, Muaf>;
struct Ogrenci {
    int numara; std::string isim;
    NotTuru not_durumu;  // variant ile esnek not
};
class YeniDepo {
    std::map<int, Ogrenci> kayitlar; // numara ile indeksli
public:
    bool ekle(const Ogrenci& ogr) {
        auto [it, eklendi] = kayitlar.emplace(
            ogr.numara, ogr);
        return eklendi;
    }
    // O(log n) arama + optional donus
    std::optional<Ogrenci> bul(int numara) const {
        if (auto it = kayitlar.find(numara);
            it != kayitlar.end())
            return it->second;
        return std::nullopt;
    }
};
```

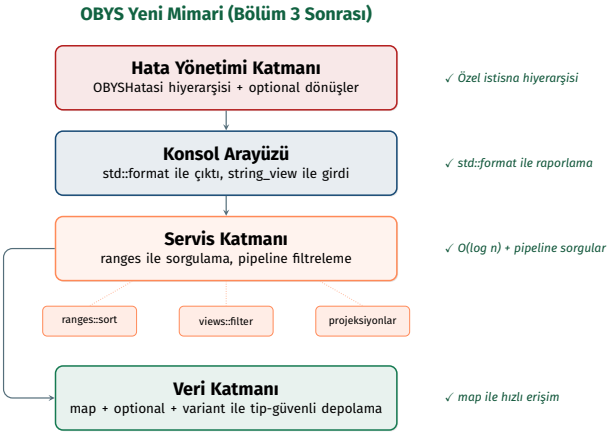
Servis Katmanı: ranges, string_view, format

```
class OgrenciServisi {
    YeniDepo depo;
public:
    // string_view parametre + ranges pipeline
    void basarili_listele(std::string_view bolum,
                          double esik = 60.0) const {
        auto tumu = depo.tumunu_al();
        auto rapor = tumu
            | std::views::filter([&](const Ogrenci& o) {
                  return /* bolum kontrolu */; })
            | std::views::filter([&](const Ogrenci& o) {
                  return /* not kontrolu */; });
        // format ile tablo ciktisi
        std::cout << std::format("{:^6} | {:<20} | {:>6}\n",
                                 "No", "Isim", "Not");
        std::cout << std::format("{:-^36}\n", "");
        for (const auto& o : rapor)
            std::cout << std::format("{:>6} | {:<20} | {:>6}\n",
                                     o.numara, o.isim, /*not*/);
    }
};
```

Hata Katmanı: İstisna Hiyerarşisi Entegrasyonu

```
class OgrenciServisi {
public:
    void notGir(int numara, int not_degeri) {
        if (!depo.bul(numara))           // 1. Ogrenci var mi?
            throw OgrenciBulunamadi(numara);
        if (not_degeri < 0 || not_degeri > 100) // 2. Not gecerli mi?
            throw GecersizNot(not_degeri);
        depo.notGuncelle(numara, not_degeri);    // 3. Kaydet
    }
};

// Kullanim
try {
    servis.notGir(9999, 85);
} catch (const OgrenciBulunamadi& e) {
    std::cout << e.what() << " (No: " << e.numara() << ")\n";
} catch (const OBYSHatasi& e) {
    std::cout << "OBYS hatasi: " << e.what() << "\n";
}
```

Yeni Mimari



Kazanımlar

`map` → $O(\log n)$ arama · `optional` → güvenli "bulunamadı" · `variant` → esnek not · İstisna hiyerarşisi

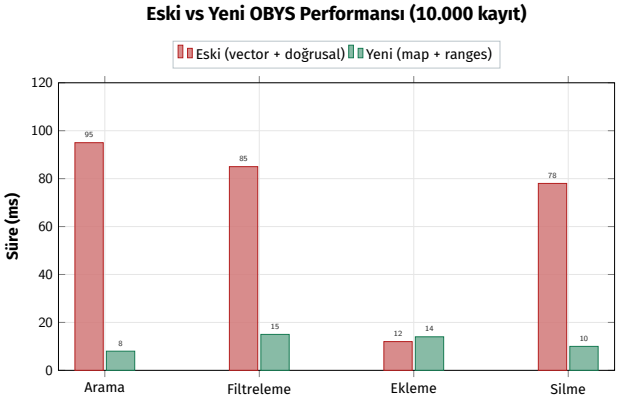
Test ve Doğrulama: assert() ile Birim Test

```
#include <cassert>
void test_depo_ekle_ve_bul() {
    YeniDepo depo;
    Ogrenci ogr{2001, "Ali", NotTuru{85}};
    // Ekleme testi
    assert(depo.ekle(ogr) == true);     // ilk ekleme basarili
    assert(depo.ekle(ogr) == false);    // cift numara reddedilmeli
    // Arama testi
    auto sonuc = depo.bul(2001);
    assert(sonuc.has_value());           // bulunmali
    assert(sonuc->isim == "Ali");
    // Bulunamama testi
    auto bos = depo.bul(9999);
    assert(!bos.has_value());            // nullopt donmeli
    std::cout << "Tum testler gecti!\n";
}
```

Test Stratejisi

Normal durum + kenar durum + hata durumu → kapsamlı doğrulama.

Performans Karşılaştırması



Eski (vector)

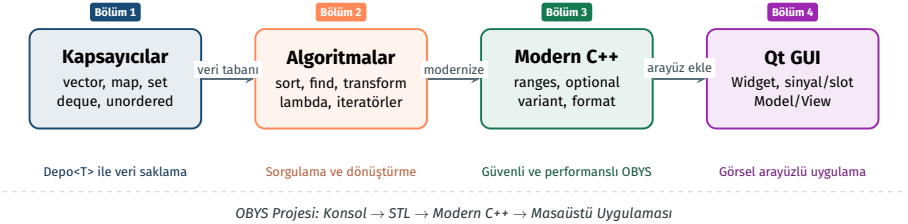
Arama: $O(n)$ · Silme: kaydır + sil

Yeni (map)

Arama: $O(\log n)$ · ∼12x hızlı

Özet

Bölüm 1–3: Büyük Resim



Yolculuk

Bölüm 1: Kapsayıcılar → **Bölüm 2:** İteratörler + algoritmalar → **Bölüm 3:** Modern C++ (güvenlik, okunabilirlik, performans)

Bu Hafta Öğrendiklerimiz

Ranges ve Views:

1. Doğrudan kapsayıcı ile algoritmalar
2. Projeksiyonlar (lambda'ya veda)
3. Tembel (lazy) views: filter, transform
4. Pipe (|) ile pipeline'lar

String İşlemleri:

5. `string_view`: kopyasız erişim
6. `std::format`: tür-güvenli biçimlendirme
7. `from_chars/to_chars`: hızlı dönüşüm
8. Veri doğrulama desenleri

Hata Yönetimi:

9. `optional`: "belki yok" durumu
10. `variant`: tür-güvenli union
11. `visit` + overloaded deseni
12. Özel istisna hiyerarşisi

OBYS Dönüşümü:

13. Üç katmanlı refactoring
14. map + optional + variant
15. assert() ile birim test
16. Performans: ∼12x iyileşme

OBYS'de Her Şey Bir Arada

```
#include <map>
#include <optional>
#include <variant>
#include <ranges>
#include <format>

struct Muaf {};
using NotTuru = std::variant<int, std::string, Muaf>;
struct Ogrenci { int no; std::string isim; NotTuru not_d; };

class OBYS {
    std::map<int, Ogrenci> kayitlar;
public:
    std::optional<Ogrenci> bul(int no) const {
        if (auto it = kayitlar.find(no); it != kayitlar.end())
            return it->second;
        return std::nullopt;
    }
    void rapor(std::string_view baslik) const {
        std::cout << std::format("{:=^50}\n", baslik);
        for (const auto& [no, ogr] : kayitlar)
            std::cout << std::format("{}: {}\n", no, ogr.isim);
    }
    auto basarili() const {
        return kayitlar | std::views::values
            | std::views::filter([](const Ogrenci& o) {
                  return std::holds_alternative<int>(o.not_d)
                      && std::get<int>(o.not_d) >= 60; });
    }
};
```

Gelecek Hafta: Qt Framework'üne Giriş

- Qt nedir? Çapraz platform GUI çerçevesi
- İlk Qt penceresi: QApplication ve QWidget
- Sinyal/Slot mekanizması: olay tabanlı programlama
- Widget'lar: buton, etiket, metin kutusu
- Düzen yönetimi: QHBoxLayout, QVBoxLayout
- OBYS: Konsoldan masaüstü uygulamasına geçiş!

Hazırlık

Kitabın Bölüm 4'ü okuyun. Qt 6 kurulumunu yapın. Lab03'teki ranges ve optional örneklerini OBYS projenize entegre edin.